🔷 PROJECT 3: IMAGE CLASSIFICATION (HEALTHCARE)

Title : Medical Image Classification using CNN (X-ray Disease Detection)

Objective

Classify medical images (X-ray/MRI) into:

- Normal
- Diseased

Tech Stack

- Python
- TensorFlow / Keras
- OpenCV

📁 Folder Structure

```
Image_Classification_Project/
├── data/
│   ├── train/
│   │   ├── normal/
│   │   └── disease/
│   ├── test/
├── model/
├── cnn_model.py
├── README.md
```

Dataset

👉 Use Kaggle dataset: Chest X-ray / Pneumonia dataset
👉 Or create dummy structure (images inside folders)

Steps

1. Load image dataset
2. Resize images
3. Normalize pixel values
4. Build CNN model
5. Train model
6. Evaluate accuracy

Full Code

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models

# Data Preprocessing
train_datagen = ImageDataGenerator(rescale=1./255)

train_data = train_datagen.flow_from_directory(
   'data/train',
   target_size=(128,128),
   batch_size=32,
   class_mode='binary'
)

# CNN Model
model = models.Sequential([
   layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
   layers.MaxPooling2D(2,2),
```

```
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train
model.fit(train_data, epochs=5)

model.save("model/cnn_model.h5")
```

Output

- Accuracy (e.g., 85–95%)
- Prediction: Normal / Disease

Documentation

- CNN introduction
- Medical imaging importance
- Model architecture
- Results & accuracy
- Healthcare impact

🔷 PROJECT 4: TIME SERIES FORECASTING

Title

Stock Price Prediction using ARIMA Model

Objective

Predict future stock prices based on historical data

Tech Stack

- Python
- Pandas
- Statsmodels

📁 Folder Structure

```
Time_Series_Project/
├── data/
│   └── stock.csv
├── model/
├── forecast.py
├── README.md
```

Dataset (stock.csv)

```
date,price
2023-01-01,100
2023-01-02,102
2023-01-03,101
```

2023-01-04,105

2023-01-05,107

2023-01-06,110

⚡ Generate Large Dataset

```
import pandas as pd
import numpy as np

dates = pd.date_range(start="2023-01-01", periods=200)
prices = np.random.randint(100, 200, size=200)

df = pd.DataFrame({"date": dates, "price": prices})
df.to_csv("stock.csv", index=False)
```

Steps

1. Load data
2. Convert date column
3. Check trend
4. Apply ARIMA
5. Forecast future values

Full Code

```
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA

# Load data
df = pd.read_csv("stock.csv")
df['date'] = pd.to_datetime(df['date'])
```

```
df.set_index('date', inplace=True)

# Model
model = ARIMA(df['price'], order=(5,1,0))
model_fit = model.fit()

# Forecast
forecast = model_fit.forecast(steps=5)
print("Future Predictions:\n", forecast)
```

Output

- Future 5-day stock predictions

Documentation

- Time Series basics
- ARIMA explanation
- Trend & seasonality
- Business applications

🔷 PROJECT 5: FRAUD DETECTION SYSTEM

Title : Credit Card Fraud Detection using Machine Learning

Objective

Detect fraudulent transactions from dataset

Tech Stack

- Python
- Pandas

- Scikit-learn

Folder Structure

Fraud_Detection_Project/

├── data/

│ └── transactions.csv

├── model/

├── fraud_model.py

├── README.md

Dataset (transactions.csv)

transactionId,amount,type,isFraud

1,500,online,0

2,10000,transfer,1

3,200,shopping,0

4,15000,transfer,1

5,300,online,0

6,25000,transfer,1

7,400,shopping,0

8,12000,transfer,1

⚡ **Generate Large Dataset**

```
import pandas as pd
import numpy as np

data = []

for i in range(1000):
```

```
    amount = np.random.randint(100, 50000)
    fraud = 1 if amount > 10000 else 0
    data.append([i, amount, fraud])

df = pd.DataFrame(data, columns=["transactionId","amount","isFraud"])
df.to_csv("transactions.csv", index=False)
```

Steps

1. Load dataset
2. Data preprocessing
3. Split data
4. Train model
5. Evaluate accuracy

Full Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load data
df = pd.read_csv("transactions.csv")

X = df[['amount']]
y = df['isFraud']

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
# Model
model = LogisticRegression()
model.fit(X_train, y_train)

# Accuracy
print("Accuracy:", model.score(X_test, y_test))
```

Output

- Fraud vs Non-Fraud prediction

Documentation

- Fraud detection concepts
- Imbalanced data problem
- Model performance
- Banking use cases